

Migración de GEOUNED a pyOCC

Informe técnico del refactor del backend CAD

Proyecto GEOUNED — rama `georeverse-migration`

6 de septiembre de 2026

Proyecto	GEOUNED-org/GEOUNED
Rama	<code>georeverse-migration</code>
Motor por defecto	<code>ocp</code> (OCP / pybind11)
Motores soportados	<code>freecad · occ · ocp</code>
Alcance	CadToCsg (GEOUNED) + CsgToCad (GEORReverse)
Fuente	registro de trabajo <code>CLAUDE.md</code>

3 / 3 motores CAD con la suite en verde | **0** partículas perdidas (corpus `test_models`, 143 ejec. d1suned) | **93,6 %** tallies dentro de 2σ / **0 %** por encima de 3σ | **0** imports directos de `Part/FreeCAD` en GEOUNED

Índice

1. Resumen ejecutivo	1
2. Contexto y motivación	2
2.1. El problema que originó la migración	2
2.2. Decisión: <code>pythonocc-core</code> , no <code>CadQuery</code>	2
3. Arquitectura: el paquete <code>geo</code>	2
3.1. Intento 1 (retirado): la ABC <code>GeometryBackend</code>	2
3.2. Intento 2 (actual): el paquete <code>geo</code>	3
3.3. Diseño de las clases <code>geo</code>	3
4. Desacoplamiento de FreeCAD en GEOUNED	4
4.1. Unificación de representaciones duplicadas	4
4.2. Dos ámbitos de numeración de superficies	4
5. Reorganización de los <code>writers</code>	4
6. Campañas de verificación	5
6.1. Verificación de <code>check_sign</code> punto a punto	5
6.2. Chequeo estocástico de volumen MCNP	5
7. Correcciones de geometría	6
8. Migración pyOCC por fases	8
8.1. Tercer motor: OCP (pybind11)	8
9. Migración de GEORReverse	8

10.Detección y reparación de defectos CAD	9
11.Estado actual y trabajo pendiente	10
11.1. Estado consolidado	10
11.2. Abierto	10
12.Anexo: entorno y herramientas	11

1. Resumen ejecutivo

GEOUNED convierte geometría CAD en geometría CSG (y a la inversa) para los códigos de transporte Monte Carlo **MCNP**, **OpenMC**, **PHITS** y **Serpent**. Históricamente dependía por completo de la API `Part` de FreeCAD y, en particular, de `BOPTools.SplitAPI.slice` para descomponer sólidos. Ese acoplamiento se ha eliminado.

El trabajo realizado se agrupa en cinco bloques:

- **Aislamiento del kernel.** Todo el código que toca FreeCAD/OCCT vive ahora en un único paquete, `geo`, tras una interfaz de nombres estable. Ningún otro fichero de GEOUNED importa `Part` ni `FreeCAD`.
- **Tres motores geométricos.** `geo` tiene tres implementaciones intercambiables mediante la variable de entorno `GEOUNED_CAD_ENGINE`: FreeCAD (`freecad`), `pythonocc-core` / SWIG (`occ`) y OCP / `pybind11` (`ocp`, actualmente el valor por defecto).
- **Verificación cuantitativa.** Cada corrección se ha validado con dos métodos independientes: comprobación punto a punto de `check_sign` contra geometría CAD construida de forma independiente, y un chequeo estocástico de volumen ejecutando el modelo MCNP resultante en `dlsuned` (`tally` $\approx 1,0$).
- **Depuración geométrica dirigida por datos.** Decenas de bugs reales — muchos preexistentes a la migración — encontrados y corregidos siguiendo trazas concretas: signos AND/OR en superficies compuestas, degeneraciones de tangencia en `Gsplit`, clasificación de superficies, escalado de tolerancias y *slivers* residuales.
- **Reparación de defectos CAD.** Una capa nueva en tiempo de carga que detecta sólidos corruptos (aristas patológicamente cortas, anillos de contorno duplicados, cortes fantasma) y los repara automáticamente cuando es rápido y fiable, o los marca sin traducir cuando no lo es.

El pipeline inverso, **GEORreverse** (`CsgToCad`), que inicialmente quedó fuera de alcance por su dependencia dura del formato `.FCStd`, ha recibido su propia migración en la rama actual: es engine-switchable, exporta STEP real vía XCAF bajo los motores `pyOCC` y colorea cada sólido por material.

Estado a fecha del informe: los tres motores pasan sus suites; el corpus de referencia `Solidos/test_models` se traduce sin partículas perdidas y sin fallos por encima de 3σ ; `occ` y `ocp` producen resultados idénticos sobre todo el corpus. Quedan abiertos varios puntos de **GEORreverse** y algunas superficies exóticas sin portar (sección 11).

2. Contexto y motivación

2.1. El problema que originó la migración

El `Cut` booleano de FreeCAD puede fallar de forma **silenciosa** al partir un sólido en dos cuando la intersección del plano de corte coincide exactamente con una línea de tangencia preexistente

(por ejemplo, un plano que corta la esquina donde un cilindro ya es tangente a dos caras de un cubo). El síntoma: `Cut` devuelve el sólido original sin cortar, en lugar de lanzar un error. La causa: los fragmentos resultantes solo se tocan a lo largo de una línea (contacto de área cero), lo que produce una forma no-manifold que el algoritmo BOP de OCC no resuelve limpiamente. Pasar por una exportación/importación STEP «arregla» el problema porque el sanador de STEP vuelve a partir el compound.

La solución de fondo — construir un grafo de adyacencia de caras que excluya las aristas compartidas por más de dos caras y reconstruir un sólido por componente conexa — requiere control topológico fino que la API `Part` de FreeCAD, al ser ella misma un envoltorio sobre OCCT, no expone cómodamente.

2.2. Decisión: `pythonocc-core`, no `CadQuery`

- Se necesita control topológico de grano fino (grafos de adyacencia cara/arista, `BRepCheck_Analyzer`, `ShapeFix_Shape`, control de tolerancia sobre `BRepAlgoAPI_*` / `BOPAlgo_Splitter`) para exactamente los casos degenerados anteriores.
- Migrar a `pyOCC` *elimina* una capa (el envoltorio `Part`) en lugar de sustituirla por otra distinta (`CadQuery`).
- `CadQuery` encaja mejor en construcción paramétrica de alto nivel, no en la cirugía topológica de bajo nivel que exige la descomposición de GEOUNED.

3. Arquitectura: el paquete `geo`

3.1. Intento 1 (retirado): la `ABC GeometryBackend`

El primer diseño fue un patrón Adapter / Ports & Adapters clásico: una clase base abstracta `GeometryBackend` con dataclasses neutras (`GSolid` / `GFace` / `GEdge` / `GVector` / ...) que llevaban un objeto `native` más una referencia al backend que las produjo, y una implementación concreta `FreeCADBackend`. Todo GEOUNED llamaba a través de una única instancia inyectada (`_backend.split(gsolid, tool, tol), ...`). Se implementó por completo y se validó contra toda la suite, pero se juzgó **demasiado pesado e indirecto** — las dataclasses cargando su propio backend y la convención `_backend.method(x, ...)` — para lo que aportaba. Se eliminó por completo junto con sus tests.

3.2. Intento 2 (actual): el paquete `geo`

`src/geouned/geo/` es el **único** lugar de GEOUNED autorizado a importar `Part` / FreeCAD / `BOPTools`.

Fichero	Responsabilidad
<code>vector_geometry.py</code>	Matemática pura sin dependencia de FreeCAD: <code>GVector</code> , <code>GMatrix</code> , <code>GBoundingBox</code> y conversores.
<code>surface_geometry.py</code>	Predicados analíticos sobre descriptores de superficie (<code>is_same_*_surface</code> , <code>is_inside_*</code> , <code>find_can_plane</code> , intersecciones plano-plano y línea-línea).
<code>solid_defects.py</code>	Detección de defectos CAD a nivel de sólido completo (<code>find_short_edges</code> , <code>find_split_ring_faces</code> , <code>near_surface_pair</code>).
<code>io_utils.py</code>	Utilidades de proceso/carga: <code>GLabelNode</code> (árbol de etiquetas STEP), <code>suppress_native_stdout</code> .

Fichero	Responsabilidad
<code>_freecad_impl.py</code>	Implementación FreeCAD: clasificación de superficies/curvas, clases de topología (<code>GEdge/GWire/GFace/GShell/GSolid</code>) construidas ansiosamente con comportamiento real como métodos, constructores <code>Gmake_*</code> y operaciones <code>Gcut/Gcommon/Gfuse/Gsplit</code> .
<code>_occ_impl.py</code>	La misma superficie de nombres contra <code>pythonocc-core</code> (SWIG).
<code>_ocp_impl.py</code>	La misma superficie de nombres contra OCP (<code>pybind11</code>).
<code>__init__.py</code>	Punto único de import. Lee <code>GEOUNED_CAD_ENGINE</code> una sola vez y reexporta los nombres desde la implementación resuelta.

Principio de intercambiabilidad. La selección de motor ocurre en tiempo de import, no por inyección de dependencias. Un motor nuevo solo tiene que definir los mismos nombres de clase y función; `geo/__init__.py` elige cuál reexportar y el resto de `GEOUNED` no cambia. `GEORreverse` replica exactamente el mismo patrón con su propio `GEORreverse/Modules/__init__.py`.

3.3. Diseño de las clases `geo`

- Las caras se clasifican ansiosamente en una de 5 superficies analíticas (plano / cilindro / cono / esfera / toro) vía `Gclassify_surface`. Las superficies compuestas (`RoundCorner`, `Can`, `TCone`, `MultiPlane`, `MultiRoundCorner`, `ReversedConeCylinder`) las ensambla el propio `GEOUNED` con `Gcut/Gcommon/Gfuse`; `geo` nunca las modela.
- La clasificación de curvas es tolerante: devuelve `None` para una arista degenerada o un tipo real no soportado (p. ej. hipérbola) en lugar de lanzar, porque los sólidos se construyen ansiosamente y una arista incidental no debe abortar el sólido entero.
- `Gsplit()` devuelve un `SplitResult` que nunca puede volver vacío silenciosamente: los casos de tangencia/degeneración se resuelven internamente (reintento de tolerancia, reconstrucción no-manifold, separación por componentes) y se reportan con `degenerate_case_handled=True`.
- `utils/geometry_gu.py::FaceGu / SolidGu` — un segundo envoltorio nativo previo a `geo`, usado por el código de descomposición — ahora **heredan** de `GFace / GSolid` y solo añaden encima los extras del lado de descomposición, en lugar de duplicar la construcción.

4. Desacoplamiento de FreeCAD en GEOUNED

El cierre de los `import Part / import FreeCAD` se hizo fichero por fichero, trazando cada consumidor real antes de tocar la fuente. `core.py` (concretamente `_set_geometry_bounding_box`) fue el último reducto, precisamente porque su `BoundingBox` se consume fuera del alcance original de la migración (`void.py`, `write_files.py`). Tras su cierre, **cero ficheros** del subpaquete `GEOUNED` importan `Part/FreeCAD/BOPTools` directamente.

4.1. Unificación de representaciones duplicadas

Para «un plano» existían tres representaciones paralelas: el descriptor `geo.GPlane`, la clase `PlaneParams` de `basic_functions_part1.py` y la clase `Plane` de `build_region/Objects.py`. Se consolidaron:

- «superficie analítica + bounding box → sólido finito recortado» para plano/cilindro/cono se unificó en tres funciones puras en `build_region/Objects.py` (`plane_polygon_from_box`, `cylinder_from_box`, `cone_from_box`), en espacio `GVector/GBoundingBox`, sin duplicación.
- Las clases `Plane/Cylinder/Cone/Sphere` de `Objects.py` colapsaron en una única `CellSurface` que envuelve un descriptor `geo`; `get_surface()` construye ahora `GPlane/GCylinder/...` directamente vía `.from_values(...)`, sin paso de traducción ni desvío nativo.
- Los `*OnlyParams` de Tier-1 pasaron a almacenar `GVector` directamente en lugar de `FreeCAD.Vector`; `check_sign_primitive` se colapsó a un despacho por diccionario contra las funciones `is_inside_*` compartidas.

Lección metodológica registrada. Un estudio estático de «quién lee este campo» es necesario pero no suficiente cuando el valor pasa por un objeto intermedio antes de llegar a la operación que le importa el tipo. Varias veces, una encuesta por `grep` concluyó que todos los consumidores toleraban `GVector`, y la suite real reveló sitios nativos que el `grep` no vio. Norma: seguir cada `AttributeError/TypeError` real de la suite completa, no solo el trazado estático.

4.2. Dos ámbitos de numeración de superficies

Cada `GeounedSurface` tiene `.bVar` (el id firmado con que se referencia) y, para tipos compuestos, `.region` (la definición booleana AND/OR) y `.components` (la relación numeración ↔ superficie). Se asignan en dos fases con numeraciones genuinamente distintas: en **descomposición** el `.bVar` es un id local desechable cuyo único fin es mantener la expresión AND/OR consistente para una construcción CAD; en **conversión** se sobrescribe con el id global canónico y deduplicado. Ambas fases *no* pueden leer la `.region` de la otra, pero sí comparten la **regla** AND/OR, extraída a funciones puras (`can_region`, `tcone_region`, `round_corner_region`, ...).

5. Reorganización de los writers

Los cuatro escritores de formato de salida tenían implementaciones casi idénticas de varios métodos. Se reorganizaron por la frontera real de convenciones de tarjeta:

- `write/mcnp_like/` — MCNP, Serpent y PHITS (los tres formatos de texto de linaje MCNP, con convenciones comunes de comentario de línea/inline y banners).
- `write/openmc/` — estructuralmente distinto (salida XML/Python, sin concepto de tarjeta de comentario).
- `write/functions.py` — hogar compartido de `get_cell_surf_summary`, `simplify_planes`, `sorted_surfaces`, usadas por los cuatro formatos.

El mixin `CommonInputWriter` (heredado por `McnpInput/SerpentInput/PhitsInput`) recoge lo duplicado solo entre los tres formatos de linaje MCNP, parametrizado por atributos de clase (`_surface_formatter`, `inline_comment_char`, ...).

Corrección real, confirmada con el usuario. La unificación de `sorted_surfaces` corrige una inconsistencia: solo MCNP aplicaba `Surfaces.IndexOffset` al reetiquetar cada `bVar`; Serpent/OpenMC/PHITS se lo saltaban. Ahora los cuatro formatos usan la versión de MCNP. El efecto es actualmente latente (`IndexOffset` es 0 salvo que `settings.startSurf` se cablee, cosa que también se hizo en esta migración) pero deja de ser una divergencia entre formatos.

6. Campañas de verificación

6.1. Verificación de `check_sign` punto a punto

Para cada superficie compuesta encontrada en un modelo, se construye un sólido CAD de referencia *independiente* por la misma vía que usa `Gsplit` para construir esa composición, se muestrean puntos aleatorios estrictamente dentro de la caja de construcción y se compara `check_sign(punto, superficie)` contra `gsolid.is_inside(punto)`.

Tipo compuesto	Resultado	Notas
RoundCorner	300/300	Múltiples instancias en decenas de ficheros.
Can (Fwd / Rev)	300/300	Un artefacto de muestreo (padding de bounding box) se identificó y corrigió: muestrear dentro de la caja de construcción, no de la del sólido resultante.
TCone (Fwd / Rev)	300/300	Reveló un bug real en <code>TCone_region</code> (ignoraba <code>p1_configuration/p2_configuration</code>), corregido.
MultiRoundCorner	300/300	17 instancias en 13 ficheros. Primera validación end-to-end de este tipo.
MultiPlane	300/300	Reveló que <code>multiplane()</code> nunca podía detectar un <code>MultiPlane</code> (comparación instancia-vs-clase), corregido.
ReversedConeCylinder	parcial	Estructuralmente inalcanzable desde el escaneo de descomposición; verificado desde el lado de conversión con una discrepancia real en <code>hylife-v06 solid113</code> (271/300), ligada a un desajuste conocido de <code>GEORreverse</code> .

6.2. Chequeo estocástico de volumen MCNP

Pregunta distinta y más end-to-end: ¿un modelo MCNP *completamente traducido*, ejecutado de verdad, reproduce los **mismos volúmenes** que los sólidos CAD originales? Se exporta con `volSDEF=True` (fuente fotónica isótropa desde una esfera envolvente + tally F4 de longitud de traza en cada celda, normalizado por el volumen CAD), se corre en `MODE P + VOID` en `dlsuned` y el resultado esperado del tally es exactamente **1,0**. Una desviación significativa ($> 3\sigma$) señala que los signos de superficie escritos no reconstruyen el mismo sólido.

Corpus Solidos/test_models (sin Big_*)	Valor
Ficheros que traducen	133 / 138
Tallies dentro de 2σ de 1,0	93,6 %
Tallies marginales ($2-3\sigma$, ruido esperado)	6,4 %
Fallos reales ($> 3\sigma$)	0 %
Ficheros con partículas perdidas	0 / 143 ejec.
Divergencia <code>occ</code> vs <code>ocp</code> sobre el corpus	byte a byte idéntica

Los 5 ficheros que no traducen son defectos CAD genuinos que la capa de detección (sección 10) marca correctamente, más un caso de crash nativo aceptado como limitación (`ConeSphere.stp` bajo `occ/ocp`).

7. Correcciones de geometría

Decenas de bugs reales, la mayoría preexistentes a la migración (confirmado contra el último commit pre-migración), encontrados siguiendo trazas concretas — nunca por conjetura — y verificados antes y después. Agrupados por naturaleza:

Categoría	Ejemplos representativos	Impacto
Signos AND/OR en superficies compuestas	<code>can_region</code> rama cono + plano-ápice Reversed usaba AND donde el signo físico exige OR (dos instancias); <code>TCone_region</code> sustituía la configuración real por un valor único según orientación; <code>round_corner_region</code> aplicaba <code>fwd_cyl</code> dos veces en la rama <code>p1 == p2</code> ; <code>cone_apex_plane</code> doble-compensaba el caso Reversed; <code>add_reversedCC</code> rediseñado (<code>plane_region AND (s1 AND ... AND sn)</code>) y su <code>AddPlanes</code> pasó de OR incondicional a AND; <code>AdjacentMultiplanePlanes</code> pasó a lista-de-listas (AND dentro de grupo, OR entre grupos).	Volumen CSG erróneo con expresión booleana válida; en varios casos «fugas» que engullían medio modelo. Verificado con <code>d1suned</code> tras cada corrección.
Degeneración / tangencia en Gsplit	Fallback de corte coaxial cono-cono (<code>_try_coaxial_cone_split</code>): reconstrucción analítica del círculo exacto de intersección, pre-split de esa arista y reintento con escalado de tolerancia difusa; el <code>splitTolerance</code> por defecto implícito pasó de 0,0 a 10^{-4} bajo <code>occ/ocp</code> (OCCT 7.9.3 es más sensible a la near-tangencia); <code>_repair_non_manifold_solid</code> ahora valida su propia reconstrucción (validez + conservación de volumen) antes de fiarse de ella.	Descomposición que devolvía silenciosamente 1 pieza en vez de 2 o 3; fragmentos con volumen inflado.
Clasificación de superficies	<code>is_same_plane_surface</code> comparaba el offset de dos planos con ejes antiparalelos como <code>d1 == d2</code> en vez de <code>d1 == -d2</code> ; <code>convex_planes</code> tenía un test de signo muerto (<code>cross.dot(ref)</code> siempre 0) y no normalizaba <code>atan2</code> a $[0, 2\pi)$; <code>multiplane()</code> comparaba instancia contra clase (<code>Gclassify_curve(e)</code> <code>is GLine</code>), haciendo <code>MultiPlane</code> indetectable; <code>is_closed_cylinder_cone</code> aceptaba una cara que no era un cilindro cerrado de $360^\circ \rightarrow$ invariante topológica real por <code>winding</code> (<code>_is_closed_by_winding</code>); <code>cyl_plane_region_conf</code> usaba la cara semilla para ambos extremos de un cilindro partido.	Clasificación de caras a tipo compuesto equivocado; tallies de 0,0 y desviaciones de cientos de σ , todas cerradas.
Tolerancias	<code>min_area</code> nunca estaba cableado a <code>order_plane_face</code> ni a los generadores de candidatos; nuevo <code>min_face_width + CharacteristicWidth</code> (ancho característico de una cara vía radios de giro) para detectar <i>slivers</i> «estrechos pero de área suficiente», portado a los tres motores; <code>Tolerances.scaled(volume)</code> escala <code>min_area/min_face_width</code> hacia abajo para fragmentos descompuestos diminutos; <code>eligible_plane</code> ignoraba las tolerancias configuradas por el usuario.	Planos sliver espurios entrando en la definición de celda $\rightarrow$ partículas perdidas.
Slivers residuales de corte booleano	<code>other_face_edge</code> con modo <code>skip_slivers</code> para atravesar caras sliver y encontrar la cara real; filtro de slivers en <code>merge_same_surface_faces</code> antes del walk $O(n^2)$ de adyacencia; <code>sort_range</code> con fallback «sin progreso» ante fragmentos unidos solo módulo 2π (era recursión infinita).	Detección de Can/RoundCorner fallando o colgándose sobre geometría con artefactos de corte.

Categoría	Ejemplos representativos	Impacto
Cross-engine	El flag de ShapeUpgrade_UnifySameDomain que provoca un crash nativo es <i>opuesto</i> entre OCP y pythonocc-core sobre la misma geometría OCCT 7.9.3; <code>get_join_cone_cyl</code> usaba <code>.Index</code> donde un shell fusionado tiene <code>.Indexes</code> (solo freecad); el LAPACK de MKL entraba en conflicto con <code>occt/tbb</code> en el mismo proceso → cambio a OpenBLAS en <code>pyoccenv</code> y <code>ocpenv</code> .	47 tests crasheando en <code>freecad/occ</code> por un <code>AttributeError</code> ; access violations nativos.
No geométricos, encontrados de camino	<code>generic_split</code> hacía doble-wrap de un <code>GSolid</code> ya envuelto → la descomposición era silenciosamente un no-op; <code>check_intersection</code> con early-exit por <code>Gdistance</code> clasificaba sólidos totalmente anidados como disjuntos → la celda de vacío del enclosure no excluía las esferas dentro (<code>w_encl.stp</code> , tally 0,0); <code>expand_regions_to_integer</code> sin guard de <code>bool</code> , guard que los cinco escritores de celda necesitaban y solo MCNP tenía.	Bugs de correctitud que degradaban la descomposición o la salida de forma invisible.

8. Migración pyOCC por fases

La migración a pyOCC se abordó en fases, cada una con un criterio de continuación explícito antes de escribir código a gran escala.

- Fase 1 — Scaffolding y prueba go/no-go.** Selección de motor vía `GEOUNED_CAD_ENGINE`; `geo/__init__.py` importa de `_freecad_impl` o del nuevo `_occ_impl`. Solo se implementó lo mínimo para probar `Gsplit` contra el caso `rev_pipe.stp`, ya caracterizado como roto en FreeCAD. Resultado: `BOPAlgo_Splitter` (OCCT 7.9.3) devuelve **3 sólidos válidos** en el primer intento sin reparación, donde FreeCAD devolvía 1 sólido inválido. Señal fuerte para continuar.
- Fase 2 — Port completo.** Los 5 descriptores de superficie y 4 de curva, `Gclassify_surface/Gclassify_curve`, `GEdge/GWire/GFace/GShell/GSolid` completos (incluyendo `find_interior_point`, `faces_sharing_edge`, `fix/refine`), todos los `Gmake_*` y las operaciones booleanas. Test nuevo `tests/geo/test_occ_impl.py`: 39/39 contra geometría real con valores esperados conocidos.
- Fase 3 — Pipeline de integración.** `tests/test_cadtocsg.py` pasa 50/50 bajo `GEOUNED_CAD_ENGINE=occ`, idéntico al baseline FreeCAD, tras cerrar 6 huecos reales (entre ellos `Gload_step_labels` vía XCAF, un bug de periodicidad en `get_join_cone_cyl` alcanzable también bajo FreeCAD, y el fallback de `GeomAPI_ProjectPointOnSurf`).
- Fase 4 — Corpus Solidos/.** Escaneo diferencial de 98/99 ficheros contra un baseline FreeCAD fresco; 7 huecos más cerrados. `rev_pipe.stp` confirmado: FreeCAD falla la ruta completa `decompose+convert` (`FuseEdges`), `occ` tiene éxito con `RoundC=2`, `MultiRoundC=1`.

8.1. Tercer motor: OCP (pybind11)

Añadido junto a FreeCAD y pythonocc-core, no como reemplazo. OCP es un binding pybind11 de OCCT más directo y activamente mantenido que el SWIG de pythonocc-core. Validación go/no-go contra `rev_pipe.stp` (2 piezas, volumen conservado), port completo estructuralmente copiado de

`_occ_impl.py` y retargeteado (con diferencias de API confirmadas en vivo: sufijo `_s` en métodos estáticos, sin `.DownCast()`, orden de argumentos invertido en `ShapeUpgrade_UnifySameDomain, ...`). `tests/geo/test_ocr_impl.py` 39/39 al primer intento; `tests/test_cadtocsg.py` 50/50; `tests/test_csgtocad.py` 2/2. Ambos motores pyOCC se desarrollan en paralelo: una corrección en uno se porta al otro.

El «gap» de rendimiento, contado con honestidad. En el caso motivador (`hyllife-v06.stp` solid 17), FreeCAD tardaba 2,3s y `pythonocc-core` 19,2s. Se atribuyó primero al coste por-llamada del binding SWIG (real: `pythonocc-core` expone la API cruda de OCCT sin la capa de conveniencia en C++ que FreeCAD sí tiene). OCP en ese mismo caso: 18,4s — solo $\sim 4\%$ más rápido, no el 2–6× de los micro-benchmarks de conversión de puntos. La causa real resultó ser **otra**: `splitTolerance` con valor implícito 0,0 hacía que OCCT 7.9.3 computara y luego descartara un split degenerado de 15 fragmentos. Con el default corregido a 10^{-4} bajo `occ/ocr`, el mismo caso baja a **1,2s** — más rápido que FreeCAD.

9. Migración de GEORreverse

GEORreverse (`CsgToCad`) reconstruye CAD a partir de CSG y depende históricamente del formato de documento `.FCStd` de FreeCAD, no solo de su kernel geométrico. Inicialmente quedó permanentemente fuera de alcance; en la rama actual (`georeverse-migration`) ha recibido el mismo tratamiento que el pipeline directo.

- **Desacoplamiento.** El núcleo de GEORreverse (`Objects.py`, `buildSolidCell.py`, `splitFunction.py`, `buildCAD.py`, `MCNPinput.py`) importa `geo` directamente, igual que el pipeline directo. Un módulo `matrix_utils.py` recoge el puente `numpy` ↔ matriz nativa. `BOPTools.SplitAPI.slice` → `Gsplit`; `FuseSolid` (triplicado byte a byte) → un único `fuse_solids` con la comprobación de invariancia de volumen de `GSolid.refine()`.
- **export_cad independiente del motor.** `export_cad(output_filename="", format="stp")` acepta un `str` o una lista de formatos, valida contra los formatos soportados por el motor y despacha por `CAD_ENGINE`. Cambio de comportamiento real: el `.FCStd` ya no se escribe incondicionalmente, solo cuando se pide `"fcstd"` explícitamente.
- **Simetría FreeCAD/pyOCC completa.** `GEORreverse/Modules/__init__.py` despacha entre `_freecad_impl.py` (export vía documento FreeCAD, `makeTree`) y `_ocr_impl.py/_occ_impl.py` (export real vía XCAF: `STEPCAFControl_Writer`, jerarquía `Universe` → `Material` → `Cell` con nombres y anidamiento). Las 6 superficies cuádricas exóticas se reorganizaron en un paquete `geo_quadrics` con el mismo patrón de despacho.
- **Color por material (motor occ/ocr).** Cada sólido del STEP exportado se colorea por su valor `MAT` vía XCAF: mismo color para el mismo material, paleta `tab10` hasta 10 materiales y rotación por ángulo áureo para más, sin colisiones. El color en el motor FreeCAD queda fuera de alcance (su API de color vive en `Gui.ViewProvider`, sin ruta `headless`).
- **IGES descartado.** Se verificó en vivo que `IGESControl_Writer` no preserva sólidos a través de IGES en esta build de OCCT (un sólido vuelve como caras planas recortadas sin topología de shell). Se abandonó importación y exportación IGES.

Verificación: `tests/test_csgtocad.py` es engine-aware y pasa 2/2 bajo los tres motores contra aserciones de volumen reales por sólido ($\pm 10^{-6}$). Bajo `occ/ocr` quedan 2 fallos preexistentes (GEORreverse produce 2/1 sólidos donde se esperan 4/5 en el STEP de `cylinder_box`), confirmados no introducidos por esta migración y no ligados al trabajo de `Gsplit`.

10. Detección y reparación de defectos CAD

Una capa nueva en tiempo de carga (`Gload_and_process_step` → `Gcheck_and_repair`) que, para cada sólido: aplica un `fix(1e-6)`, comprueba defectos conocidos y, si los hay, intenta repararlos con el método más apropiado; solo si la reparación falla se consulta el modo elegido por el usuario. El parámetro `corrupted_solids` tiene dos modos, "stop" (por defecto) y "remove"; el antiguo parámetro `invalid_solids` se fusionó aquí.

Defecto	Detección	Reparación	Estado
Anillo de con-torno partido (micro-recorte duplicado / escalón colapsado, variante esférica)	<code>find_split_ring_faces</code> : cara analítica curva a escala sliver que además toca una arista patológicamente corta. <code>count_split_ring_pairs</code> cuenta pares de aristas circulares casi coincidentes y concéntricas.	<code>Gcollapse_split_rings</code> (ocp/occ): quita las caras «riser», re-cose el shell, rehace el sólido. Aceptación por 3 comprobaciones: validez + $ \Delta V /V < 3 \times 10^{-4}$ + los pares de anillos deben decrecer.	reparado <code>barrel</code> <code>bottom.stp</code> → <code>tally</code> 0,9997
Escalón cilíndrico colapsado (familia beltline)	Mismo detector de anillo partido dispara; <code>near_surface_pair</code> identifica dos cilindros de radio casi igual bridged por una banda sliver.	<code>Gsliver_heal</code> (ocp/occ), versión 0: quita las caras sliver y la menor del par cercano, re-recorta la cuadrada liberada a su radio real, re-cose <i>al final</i> . Gate: validez + $ \Delta V < 5 \times 10^{-4}$.	reparado <code>LR.stp</code> → <code>tally</code> 0,9989 (era 0,0 / 24 partículas perdidas)
Corte fantasma (dos regiones volumétricas que solo se tocan por aristas — la motivación fundacional de la migración)	<code>_separate_edge_joined_components</code> cuenta las aristas no-manifold (compartidas por $\neq 2$ caras) → grafo de adyacencia de caras solo por aristas manifold → si hay ≥ 2 componentes conexas y las aristas no-manifold eran la conexión portante.	Quita la componente por separado (sin caras donantes: un corte fantasma real ya lleva el contorno cerrado de cada región). Gate: ≥ 2 piezas, todas válidas, suma de volúmenes conservada a 10^{-6} .	resuelto <code>ocp/occ</code> · <code>Piece_1+Tool_1</code> → 2 sólidos válidos (era 1 sólido inválido fusionado). FreeCAD no puede hacerlo.
Wire de cara malformado (<code>UnorientableShape</code>)	<code>check_solid_defects</code> reporta topología inválida; <code>BRepCheck_Analyzer</code> marca la cara, ninguna arista sliver.	Ninguna. Se probaron <code>ShapeFix_Shape</code> , <code>ShapeFix_Face</code> , partición del wire y reconstrucción — todas fallan o desvían el volumen > 1,7 %.	<i>sin reparación</i> <code>L4_support.stp</code> movido a <code>Solidos/Detected_corrupted</code> excluido de conversión

Un cambio relacionado: `check_solid_defects` ya no rechaza un sólido por una arista sliver sobre una cara *bien formada* (`CharacteristicWidth` normal) — el pipeline de descomposición ya tolera aristas sliver en caras sanas, y ningún reparador quita tal arista sin una cara sliver donde anclarse. Las recetas de defectos se mantienen en un catálogo vivo (`reference_cad_defect_recipes.md`): síntoma observable, firma de detección general, causa raíz, reparación si es rápida y fiable, y un fixture STEP mínimo con verificación dlsuned.

11. Estado actual y trabajo pendiente

11.1. Estado consolidado

- Los tres motores pasan sus suites: **freecad** 158/158, **occ** 128/128, **ocp** 128/128 (+2 fallos preexistentes de GEORreverse bajo los motores pyOCC).
- **Corpus Solidos/test_models**: 93,6 % < 2σ , 0 % > 3σ , 0 partículas perdidas en 143 ejecuciones dlsuned. **occ** y **ocp** byte a byte equivalentes sobre todo el corpus.
- **ocp** es el motor por defecto cuando **GEOUNED_CAD_ENGINE** no está definida.

11.2. Abierto

Ítem	Naturaleza
Discrepancia de volumen en el round-trip de GEORreverse sobre hylife-v06.stp celda 1 (CAD reconstruido $1,401\times$ vs tally dlsuned $1,119\times$). test_csgtocad.py 2 fallos bajo occ/ocp (2/1 sólidos donde se esperan 4/5).	GEORreverse, no trazado hasta el fondo. GEORreverse, pre-existente, acotado a Objects.py::CellObj.buildShape .
AdjacentMultiplanePlanes necesita extenderse de RevCC a MultiRoundCorner. Las 6 cuádricas exóticas (Gmake_elliptic_cone , Gmake_hyperboloid , ...) son stubs en los backends pyOCC de GEORreverse.	Mejora de cobertura, pendiente desde 2026-08-21. Port pendiente; sin fixture que las ejerce todavía.
Big_complex_cell/modelcell_cut1.stp y modelCell_670000.stp pierden partículas.	No trazado; excluidos de los escaneos de corpus por conveniencia.
hylife-v06.stp round-trip completo de 372 sólidos: descomposición lenta en meta_list[45] (fragmentos de cara duplicados, coste $O(n^2)$ en same_faces). ConeSphere.stp : crash nativo (access violation) bajo occ/ocp en ShapeUpgrade_UnifySameDomain ; traduce limpio bajo freecad . Reorganización y dedup del árbol de fixtures Solidos/ .	Causa raíz identificada, no optimizado, despriorizado por el usuario. <i>limitación aceptada</i> Sin knob de OCCT que lo evite; documentado en los docstrings. Parcial (test_models/ + algunos duplicates_removed/).

Prioridad explícita del usuario: terminar de limpiar los bugs conocidos del pipeline directo (GEOUNED) antes de retomar los ítems de GEORreverse.

12. Anexo: entorno y herramientas

Entorno conda	Binding	OCCT · Python	Uso
(python por defecto)	FreeCAD 1.1.1	7.8.1 · 3.11	GEOUNED_CAD_ENGINE=freecad
pyoccenv	pythonocc-core (SWIG)	7.9.3 · 3.12	GEOUNED_CAD_ENGINE=occ
ocpenv	OCP (pybind11)	7.9.3 · 3.12	GEOUNED_CAD_ENGINE=ocp (por defecto)

- **FreeCAD** y **pyOCC** no coexisten en un mismo proceso (Module use of python311.dll conflicts): FreeCAD trae Python 3.11, los entornos **pyOCC** usan

3.12. Cualquier prueba del lado pyOCC usa el `python.exe` del entorno correspondiente y nunca mezcla en la misma invocación.

- Los entornos pyOCC necesitaron cambiar el proveedor de BLAS/LAPACK de MKL a Open-BLAS (`conda install -c conda-forge -override-channels "libblas=***openblas" ...`): el LAPACK de MKL entra en conflicto con `occt/tbb` en el mismo proceso y provoca un `access violation` en `numpy.linalg`.
- El chequeo de volumen MCNP se ejecuta con **dlsuned** (variante MCNP interna) a través de WSL. El pipeline completo, scripts y resultados están archivados bajo el WSL (Ubuntu-22.04) en `~/work/taller/SolidTestMCNP/scripts/` (conversión y `dlsuned` en paralelo por motor: `run_all_conversions*_parallel.py`, `run_test_models*_dlsuned.sh`, `analyze_test_models*.py`).
- Convención del árbol de trabajo: `Solidos/` solo STP; `Solidos/test_models/` el set curado de regresión; `../calculos_CLAUDE/` toda ejecución de cálculo; `Solidos/BadCAD_decomposition/`, `Solidos/Detected_corrupted/`, `Solidos/no_convierte/`, `Solidos/convierte_bad_volume/` carpetas de triaje.

Informe sintetizado a partir del registro de trabajo `CLAUDE.md` del proyecto `GEOUNED`. Rama `georeverse-migration`. Fecha: 2026-09-06.

Preferencia de código del proyecto: la comunicación es en español; todo el código, comentarios, docstrings y nombres de identificadores en inglés.